

An analysis of malicious behaviors of open-source packages using dynamic analysis

Thanh-Cong Nguyen¹, Duc-Ly Vu², and Narayan C. Debnath²

¹ University of Information Technology, Ho Chi Minh City, Vietnam
20521143@gm.uit.edu.vn

² School of Computing and Information Technology, Eastern International
University, Binh Duong, Vietnam
ly.vu@eiu.edu.vn, narayan.debnath@eiu.edu.vn

Abstract. There has been an increasing number of malicious open-source packages in recent years. A recent backdoor attack on the Linux *xx* utility has highlighted the importance of security checks on open-source packages, especially popular ones. While major security scanners focus on identifying vulnerabilities (CVEs) in open-source packages, there are very few studies on malware analysis techniques for them. Similar to traditional malware analysis, there are two types of analysis for open-source packages: static and dynamic analysis. Static analysis techniques mainly focus on analyzing the source code of a package while dynamic analysis techniques execute the code in an isolated environment. Dynamic analysis techniques seem more promising than static analysis techniques, as they can expose packages' behaviors at runtime. However, current dynamic analysis tools (e.g., *package-analysis*) make minimal effort to provide insight into the behaviors of open-source packages. In this paper, we attempt to analyze the dynamic behaviors of open-source packages on popular package repositories, including npm, PyPI, RubyGems, Packagist, and crates.io. We also analyze the discrepancies in behaviors between benign and malicious packages at runtime, which is helpful in building rules for malware detection. Our study finds that malicious packages perform a significantly higher number of domain communications and command executions. Malicious packages use simple techniques for malicious operations such as *base64* or *curl*.

Keywords: Dynamic malware analysis, Open-source malicious packages, Open-source software security, Software supply chain Security, Software supply chain attacks

1 Introduction

In modern software development, developers usually use third-party open-source packages or libraries from language-based package repositories (e.g., PyPI for Python). This practice saves developers' time and improves development velocity. Besides the benefits of using open-source packages, there are security risks in package repositories. For example, attackers could implant malicious code in

the source code repository of a popular package to infect its users. The recent *xz* attack magnifies the importance of scanning open-source code before using it [8].

Researchers and commercial organizations have proposed various techniques and developed tools for scanning malicious packages. Basically, the tools can be categorized into: static and dynamic malware analysis tools. Static analysis tools only examine package information (e.g., source code or metadata) without executing them while dynamic analysis performs package execution in an isolated environment. While static analysis tools are fast and easy to implement, they seem to be ineffective against anti-analysis techniques such as code obfuscation [15]. In addition, statically analyzing packages could introduce many false positives [29], one of the static malware detection tools called *OSSGadget* [14] even highlights this limitation on their GitHub page.

Dynamic malware analysis techniques, on the other hand, run the code in an isolated environment, typically a sandbox, and observe its behaviors, such as system calls or network connections. Current dynamic analysis tools for open-source packages are promising but still seem to be immature [29]. For example, *package-analysis*, a dynamic analysis tool developed by *OpenSSF*, has been used to detect malicious packages [18]. However, we notice that the tool only provides raw analysis results in JSON format, which requires extensive analytical effort. This effort involves either looking at the raw results or writing detection rules to decide whether a package is malicious. In this regard, this paper takes a step forward in mining the raw outputs of *package-analysis* and providing insights into the behaviors of benign and malicious packages in popular package repositories.

When analyzing open-source packages, researchers typically look for malicious indicators (e.g., suspicious domains or system calls) in the analysis report based on their expert knowledge to decide whether a sample is malicious. False positives can occur in the analysis, where a benign package is mistakenly flagged as malicious [29]. To prevent false positives, malware detection tools need to differentiate malicious behaviors from benign ones. To the best of our knowledge, there is no study in the literature analyzing the malicious behaviors of open-source packages using dynamic analysis.

This paper studies the behaviors and characteristics of benign and malicious open-source packages in popular package repositories, including crates.io, npm, Packagist, PyPI, and Rubygems. We have created a dataset consisting of malicious and benign packages. We then analyze the dataset to understand the differences between benign and malicious behaviors. Our analysis shows that the malicious packages perform a significantly higher number of domain communications and command executions compared to benign packages. The malicious packages, however, employ simple techniques such as base64 for data encoding and curl for exfiltrating users' information to a remote server.

In summary, this paper has the following contributions:

- A methodology for curating malicious and benign packages
- An investigation of a dynamic malware analysis tool called *package-analysis* for open-source packages

- An analysis of malicious behaviors and benign behaviors in open-source packages

2 Background

2.1 Software supply chain attacks

Software supply chain attacks occur when attackers inject malicious code into a component in the software supply chain [25]. End users can be infected by downloading or updating the software product. Ladisa et al. [13] present a taxonomy of software supply chain attacks on package managers and their countermeasures. In their paper, typosquatting and combosquatting techniques are the most popular methods to confuse end users and lure them into downloading malicious packages. Several approaches [30, 23] have been proposed to detect this type of attack.

One notable example of the software supply chain attack is the *SolarWinds* attack where attackers were able to inject malicious code into a company software's update [5]. Recently, malicious code was discovered in the upstream tarballs of *xz*, starting with version 5.6.0. The tarballs included extra .m4 files, which contained instructions for building with automake that did not exist in the repository. These instructions, through a series of complex obfuscations, extract a prebuilt object file from one of the test archives, which is then used to modify specific functions in the code while building the *liblzma* package. This issue results in *liblzma* being used by additional software, such as *sshd*, to provide functionality that will be interpreted by the modified functions [9].

2.2 Static Malware Analysis Tools

Static malware analysis techniques look for malicious patterns in the source code or metadata of a package. Although these techniques are lightweight, they cannot catch malicious code that is only executed at runtime. Moreover, static analyzers are prone to anti-analysis techniques, such as code obfuscation. There are existing static malware scanners, including the following:

- OSS Detect Backdoor [14]: an open-source tool developed by Microsoft. It contains a set of utilities for investigating different aspects of an open-source package.
- Bandit4Mal [27]: a tool developed by academic researchers at the University of Trento and SAP Security Research. This tool scans Python packages for malicious traits using AST analysis and hand-written malware detection rules [28].
- PyPI Malware Checks [31]: a tool used by PyPI to check for suspicious code lines in every package uploaded to PyPI. The tool uses a set of regular expression-based rules.
- Capslock [12]: a capability analysis command line interface (CLI) program for Go packages that informs users of which privileged operations a given package can access. Currently, Capslock only supports Go.

The above-mentioned tools normally parse the code of a package into ASTs and use a set of rules to look for malicious patterns. The study by Vu et al. [29] evaluates different static malware detection tools for open-source packages and reports that they generate many false positives. The same study suggests considering dynamic scanning techniques, in particular running code in a sandbox, for better malware detection.

2.3 Dynamic Malware Analysis Tools

Dynamic analysis tools require executing the source code of a package in an isolated environment. By running the code, these tools record execution traces of the package, such as running processes, executed commands, communicated IPs/domains, and accessed files. Although dynamic analysis tools can provide a more precise analysis, they usually take time and require setting up an appropriate environment. The following are dynamic analysis tools:

- MalOSS [10]: a tool that uses Sysdig [22] as the tracing tool to capture system call traces related to IPs, DNS queries, files, and processes.
- package-analysis [17]: an open-source dynamic analysis tool, developed by Google in 2022. The tool monitors command, file, and network activities in a sandbox called *Gvisor* (discussed later in this paper).
- package-hunter [11]: a tool that analyzes a program’s dependencies for malicious code. The tool installs the dependencies in a sandbox environment and monitors system calls executed during the installation [6].

In this paper, we choose *package-analysis* as our analysis tool because it is an open-source tool. The tool analyzes the capabilities of packages available on open-source repositories. The tool looks for behaviors that indicate malicious software: 1) *What files do they access?*, 2) *What addresses do they connect to?*, and 3) *What commands do they run?* [17]. *package-analysis* captures malicious interactions with the system as well as network connections that can be used to leak sensitive data or allow remote access using the *Gvisor* sandbox [7]. In addition, the raw outputs of *package-analysis* are available on Google BigQuery [2] enabling us to perform an in-depth analysis of the behaviors of benign and malicious packages.

3 Sandboxing

A sandbox is an isolated environment used to dynamically execute suspicious code. In this way, we can run untrusted programs in a safe environment without affecting real systems [21]. In this section, we discuss the architecture and limitations of *Gvisor*, the sandbox that underpins *package-analysis*.

Figure 1 shows the overall architecture of *package-analysis*. This architecture includes a nested container, where the sandbox uses a container inside another container. By doing so, *package-analysis* creates a safe, isolated environment for

executing suspicious code. In particular, the outer container uses a Docker image called *gcr.io/ossf-malware-analysis/analysis* to create the inner container. *Gvisor* runs on multiple architectures (as Linux does) such as x86, ARM, and Virtual Machines (VMs). *Gvisor* intercepts all system calls from sandboxed applications to the Linux kernel. Although *Gvisor* provides beneficial features, it has the following limitations:

- *Gvisor* runs in user space, so it has a lower priority than the kernel when executed.
- Currently, *Gvisor* does not provide a complete list of system calls like the kernel. In particular, *Gvisor* only supports the 211 most common system calls. For unsupported system calls, *Gvisor* will not process them and will raise exceptions.
- Applications in *Gvisor* cannot interact with the hardware of the host machine, as *Gvisor* creates a protected layer to prevent any interaction between applications and the host machine [24].

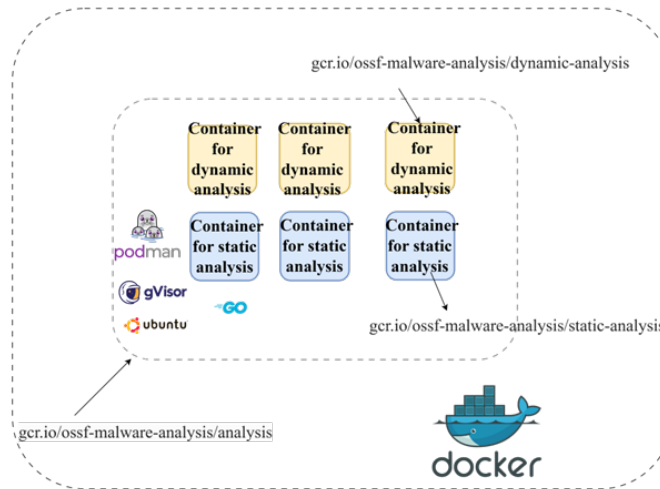


Fig. 1: Package Analysis Sandbox Architecture.

4 Data Collection

This section presents our data collection and analysis workflow. In particular, we present two datasets: malicious packages and benign packages.

4.1 Malicious Packages Collection

Figure 2 shows our data collection workflow. We collected malicious open-source packages from the following sources:

- Vulert [4]: this service provides security information (such as CVE IDs) about open-source packages in popular package repositories such as npm, PyPI, RubyGems, crates.io.
- Vulners [1]: Vulners maintains a database of software vulnerabilities. Vulners also provides Application Programming Interfaces (APIs) to search for specific software vulnerabilities.
- OSV [3] monitors open-source packages for vulnerabilities. Like Vulners, this service provides APIs to query security information about open-source packages, including the identification of malicious packages.

After obtaining malicious package names and their descriptions (e.g., behavior descriptions), we query the analysis results of the malicious packages on Google BigQuery’s *ossf-package-analysis* [2]. Next, we extract the executed commands, IP addresses, and domains for each package analysis report for further analysis (as shown in Section 5).

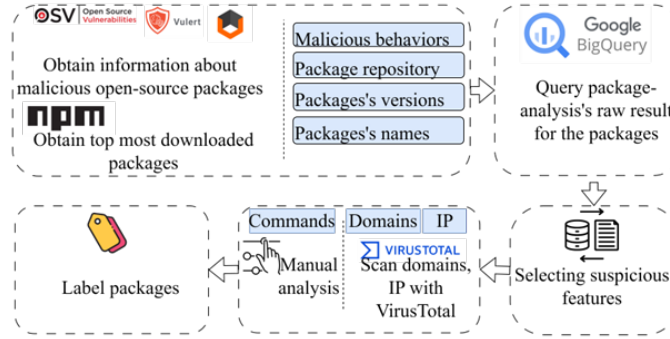


Fig. 2: Our data collection and analysis workflow.

Table 1 shows the statistics of the malicious packages we have collected. On average, each package has two versions. We notice that npm has the most packages and versions (approximately 90%) in the collected dataset. Conversely, crates.io has the fewest packages and versions. This might indicate that, at the moment, npm is the most valuable target for attackers to inject malicious code. Researchers might need to pay more attention to vetting this repository for safer use.

4.2 Benign Packages Collection

Following Zahan et al. [32] and Vu et al. [29], we collected the top 1000 most downloaded open-source packages on npm as benign packages. Note that we chose npm packages because they are the most common packages in the malicious packages dataset. In the end, we have a balanced dataset that has a fair number of packages. We then queried the raw analysis for those benign packages on

	#Packages	#Versions
crates.io	1	10
npm	1041	2293
PyPI	113	216
RubyGems	16	27
Total	1171	2546

Table 1: Statistics of collected malicious open-source packages.

Google BigQuery [2]. The raw results of the benign packages are analyzed and compared with those of the malicious packages in the next section.

5 Findings

5.1 Performance of package-analysis on open-source packages

In this section, we investigate the dataset published by OSSF on Google BigQuery called *ossf-malware-analysis* [2]. This dataset contains the live analysis of *package-analysis* on open-source packages from crates.io, npm, PyPI, Packagist, RubyGems package repositories. Table 2 summarizes the analysis performance of *package-analysis* on the packages from the supported package repositories. We can see that *package-analysis* covers most of the packages on crates.io (87.2%) while having the lowest coverage (16.37%) for Packagist. Surprisingly, although npm has the highest number of packages, only about 28% of its packages have been analyzed by *package-analysis*.

Repository	Language	#Packages in repository	#Packages analyzed by package-analysis	Ratio of analyzed packages and total packages
crates.io	Rust	144 047	125 640	87.22%
npm	Javascript	4 530 434	1 264 900	27.92%
Packagist	PHP	390 942	63 987	16.37%
PyPI	Python	535 457	287 299	53.65%
RubyGems	Ruby	197 071	31 803	16.14%

Table 2: Statistics of open-source packages on Package Analysis’s BigQuery.

Figure 3 shows the completion rates of importing and installing packages in different repositories. We observed that on average, *package-analysis* has success rates of 62.75% and 95.81% when installing and importing a package, respectively. Packages in npm and crates.io have the highest success rate when being installed and imported, respectively. However, crates.io has the lowest success rate when being installed by *package-analysis*. This indicates that installing a Rust package in crates.io is still a challenging problem.

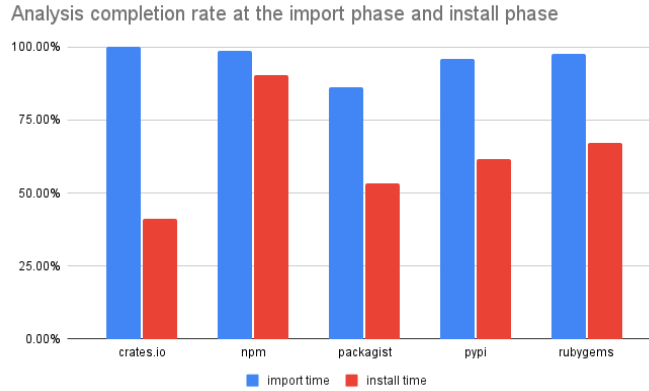


Fig. 3: Analysis completion rate of package-analysis at the import phase and install phase.

5.2 Analysis of malicious and benign packages

In this section, we report our findings on the behaviors exposed by the benign and malicious packages in our collected dataset. Table 3 shows the behaviors of malicious packages in crates.io, npm, PyPI, and RubyGems. Note that we could not find records for the packages in Packagist from our sources in Subsection 4.1. Table 3 shows that at least one malicious package from each package repository communicates with a domain associated with malicious activity. The packages in npm and PyPI also execute one or more commands associated with malicious behavior. One-third of the npm packages exhibit both domain communication and command execution behaviors.

	Communicates with a domain associated with malicious activity	Executes one or more commands associated with malicious behavior	Communicates with a domain associated with malicious activity and executes one or more commands associated with malicious behavior.
crates.io	1	0	0
npm	614	106	321
PyPI	86	5	22
RubyGems	16	0	0

Table 3: Frequency Statistics of Suspicious behaviors in Benign and Malicious Datasets.

Malicious packages can communicate with a malicious domain to download additional malware, known as *droppers*. Several reports have shown that npm and PyPI packages have been found to install Linux cryptominers, information

stealers, or Windows Trojans [19, 26, 20]. Malicious packages can download a script from a C&C server and then execute it on the victim’s system.

Dataset	#Commands	#Unique Commands	#URLs	#Unique URLs	#IP dresses	Ad- #Unique Addresses	IP
Malicious	2 845 356	533	68 677 299	934	74 584 405	682	
Benign	1 310 054	818	5 024 881	7	5 007 963	134	
Total	4 155 410	2760	73 702 180	941	79 592 368	816	

Table 4: Frequency Statistics of Malicious Indicators in Benign and Malicious Datasets.

Commands Analysis Malicious packages frequently run system commands on the victim’s systems. For example, malicious packages can use the *base64* command to encode users’ information before sending it out to a remote server. Table 4 shows the occurrences of commands, IP addresses, and URLs in the benign, malicious, and combined sets. We observe that malicious packages execute twice as many commands as benign packages. However, malicious packages often perform the same commands multiple times, which might indicate that malicious actors could reuse the code.

Table 5 shows the top ten commands executed by the malicious packages in our collected dataset. It is evident that most commands relate to information-gathering activities. It is noted the malicious packages employ simple techniques for malicious tasks, such as using *base64* for data encoding. Compared to traditional malware on Windows or Linux, malicious packages in the package repositories are much simpler, and some are distributed as Proof-of-Concepts (POCs). However, it is expected that malicious packages will evolve in both quantity and quality.

Command	#Occurences	Description	Malicious behavior
ls	87 706	Lists computer files and directories	Information gathering
bash	87 656	Starts a new bash shell	Command execution
cat	87 500	Views the contents of a file	Information gathering
dpkg-query	82 758	Shows information about dpkg packages.	Information gathering
lsb_release -a	82 758	Gets distribution-specific information.	Information gathering
base64	78 146	Encodes and Decodes data	Data hiding
/usr/bin/curl	77 026	Tranfers data using various network protocols.	Data infiltration
which	68 900	Identifies the location of executables	Information gathering
which bash	68 652	Identifies the location of the bash executable	Information gathering
tr	64 524	Translates or Deletes characters	Data hiding

Table 5: Top ten commands executed by the malicious packages in our dataset.

Compared to the top commands in the malicious packages, *ls* remains the most common command executed by the benign packages. In the benign packages, *grep* is the second most common command, used for searching and manipulating text patterns within files. Similar to the malicious packages, the benign packages also run some commands such as *uname* to obtain system information, including the name of the operating system. However, the benign packages do not frequently execute shell-related commands, such as *bash*. The *bash* command starts a new shell within the original shell, allowing attackers to run additional commands or shellcodes.

Domains and IP Addresses Analysis Malicious packages typically communicate with external servers, commonly referred to as Command and Control (C&C) servers, to receive instructions or send stolen data. In this section, we analyze the domains contacted by malicious packages for communication purposes. Figure 5 shows that the majority of the domains are flagged by a security vendor. At least 50 domains are flagged by two or more security vendors in VirusTotal. The higher the number of flags raised by security vendors for a domain, the greater the confidence that the domain is malicious.

Table 4 shows that the malicious packages communicate with significantly more domains (URLs) than benign packages - nearly 14 times as many. Furthermore, there is a much greater diversity of domains contacted by malicious packages - nearly 133 times more than those contacted by benign packages. This observation might indicate that malicious actors originate from different groups or that they frequently change their C&C servers after being detected.

Interestingly, it appears that malicious packages appear to be using Out-of-band Application Security Testing (OAST) tools when probing for Common Vulnerabilities and Exposures (CVEs). We identified the following OAST domains used in probing attempts for CVEs. Attackers may have scanned victims to find vulnerable targets. Additionally, these domains include exploits that have led attackers to install cryptominers on compromised hosts [16]. Table 6 presents the OAST domains associated with the malicious packages in our dataset. All the domains are flagged by at least two security vendors in VirusTotal. Most of the domains were flagged as malicious, malware, or phishing by the security vendors.

Besides domain names, IP addresses are among the basic features of network activity. IP addresses embedded in malicious packages usually indicate the locations of command and control servers. However, in benign packages, IP addresses may represent the locations of database servers or other legitimate services. Table 4 shows that there are nearly 15 times more IP addresses in malicious packages than in benign packages. Out of 37 423 IP addresses, 15 927 IP addresses (42.56%) were recognized as malicious by at least one security vendor in VirusTotal. Figure 4 shows that most of the IP addresses in the benign packages are in the United States. We observe that Germany has the second-largest number of IP addresses, which could suggest that malicious packages targeted users in Europe.

OAST Domain	#Flagged AVs	Labels assigned by AVs
oast.fun	11	Malicious, Suspicious, Phishing
oast.me	11	Malicious, Malware, Phishing
oast.live	10	Malicious, Malware, Phishing
oast.pro	10	Malicious, Malware, Phishing, Suspicious
oast.site	10	Malicious, Malware, Phishing, Suspicious
oast.online	7	Not Recommended, Malicious, Phishing, Suspicious
oastify.com	2	Malicious

Table 6: Out-of-band Application Security Testing (OAST) domains utilized by malicious packages during probing attempts for Common Vulnerabilities and Exposures (CVEs).

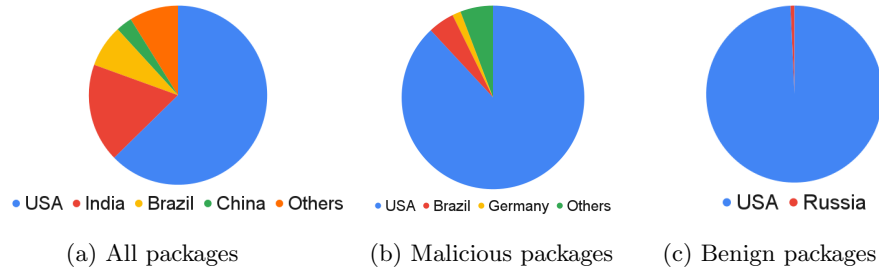


Fig. 4: Geographic locations of IP Addresses found in open-source packages.

6 Limitations and Future Work

Currently, we have only analyzed the behaviors of open-source packages in the Linux environment because *package-analysis* currently supports only a Linux sandbox. Our next plan is to extend its capability to support the Windows environment. This will require developing a Windows kernel and its utilities. In addition, engineering efforts will be necessary to improve the analysis completion rates of *package-analysis* (as shown in Figure 3), particularly in the installation phase.

In our study, we observe that *package-analysis* does not perform well during the installation phase (as shown in Figure 3). This limitation may prevent our analysis from assessing the behaviors of all the packages in the studied repositories. Therefore, our next step is to investigate the *package-analysis* logs to identify and resolve the errors.

Our analysis, based on *package-analysis*, does not indicate whether a package is malicious. Users of the tool must manually analyze the raw results it produces to make informed decisions. A promising direction is to apply machine learning techniques to the raw analysis generated by *package-analysis* on Google BigQuery [2]. For example, commands, domain URLs, and IP addresses could serve as valuable features for machine-learning-based malware detection approaches.

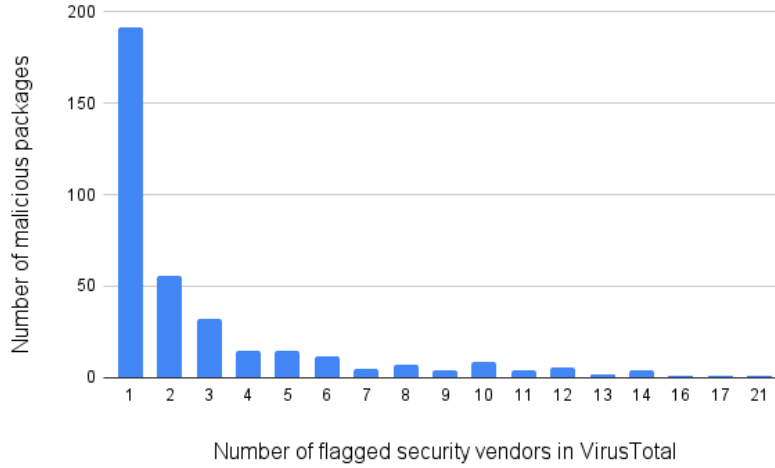


Fig. 5: Distribution of number of domains flagged by security vendors in VirusTotal

7 Conclusion

In this paper, we have taken a closer look at a dynamic analysis tool called *package-analysis*, including its sandbox technique and results. We have analyzed the raw results of *package-analysis* for open-source packages in popular repositories to identify common malicious behaviors. Our analysis indicates that malicious packages employ simple techniques such as *base64* for data encoding or *curl* for data transfer. Compared to benign packages, malicious packages exhibit significantly higher activity in command executions and domain communications.

In summary, this paper contributes to the analysis of open-source malware through dynamic analysis. This contribution could benefit security researchers in analyzing malicious open-source packages and developing detection tools.

References

1. CVE Database - Security Vulnerabilities and Exploits — Vulners.com — vulners.com. <https://vulners.com/>. [Accessed 25-04-2024]
2. Google bigquery’s ossf-malware-analysis. URL <https://console.cloud.google.com/bigquery?d=packages&p=ossf-malware-analysis&t=analysis&page=table>
3. OSV - Open Source Vulnerabilities — osv.dev. <https://osv.dev/>. [Accessed 25-04-2024]
4. Vulert: Software Composition Analysis & Vulnerability Alerts. — vulert.com. <https://vulert.com/>. [Accessed 25-04-2024]
5. Alkhadra, R., Abuzaid, J., AlShammari, M., Mohammad, N.: Solar winds hack: In-depth analysis and countermeasures. In: 2021 12th International Conference

- on Computing Communication and Networking Technologies (ICCCNT), pp. 1–7. IEEE (2021)
6. Appelt, D.: Meet package hunter: A tool for detecting malicious code in your dependencies. <https://about.gitlab.com/blog/2021/07/23/announcing-package-hunter/> (2021)
 7. gVisor Authors, T.: The container security platform. <https://gvisor.dev/>
 8. Bals, F.: What is the xz utils backdoor : Everything you need to know about the supply chain attack. <https://www.synopsys.com/blogs/software-security/xz-utils-backdoor-supply-chain-attack.html> (2024)
 9. Database, G.A.: Malicious code was discovered in the upstream tarballs of... (2024). URL <https://github.com/advisories/GHSA-rxwq-x6h5-x525>
 10. Duan, R., Alrawi, O., Kasturi, R.P., Elder, R., Saltaformaggio, B., Lee, W.: Towards measuring supply chain attacks on package managers for interpreted languages. arXiv preprint arXiv:2002.01139 (2020)
 11. GitLab: Package hunter: A tool for identifying malicious dependencies via runtime monitoring. <https://gitlab.com/gitlab-org/security-products/package-hunter> (2020)
 12. Google: Github - google/capslock: A capability analysis cli for go packages (2020). URL <https://github.com/google/capslock>
 13. Ladisa, P., Plate, H., Martinez, M., Barais, O.: Sok: Taxonomy of attacks on open-source software supply chains. In: 2023 IEEE Symposium on Security and Privacy (SP), pp. 1509–1526. IEEE (2023)
 14. Microsoft: Oss detect backdoor (2019). URL <https://github.com/microsoft/OSSGadget/wiki/OSS-Detect-Backdoor>
 15. Moser, A., Kruegel, C., Kirda, E.: Limits of static analysis for malware detection. In: Twenty-third annual computer security applications conference (ACSAC 2007), pp. 421–430. IEEE (2007)
 16. Networks, U.P.A.: Threat brief: Multiple ivanti vulnerabilities. <https://unit42.paloaltonetworks.com/threat-brief-ivanti-cve-2023-46805-cve-2024-21887/> (2024)
 17. Ossf: Github - ossf/package-analysis: Open source package analysis (2022). URL <https://github.com/ossf/package-analysis>
 18. OSSF: Package analysis: Case studies. https://github.com/ossf/package-analysis/blob/main/docs/case_studies.md (2022)
 19. Sharma, A.: 241 npm and pypi packages caught dropping linux cryptominers. <https://www.bleepingcomputer.com/news/security/241-npm-and-pypi-packages-caught-dropping-linux-cryptominers/> (2022)
 20. Sharma, A.: Attacker floods pypi with 1000s of malicious packages that drop windows trojan via dropbox. <https://blog.sonatype.com/attacker-floods-pypi-with-450-malicious-packages-that-drop-windows-trojan-via-dropbox> (2023)
 21. Sikorski, M., Honig, A.: Practical malware analysis: the hands-on guide to dissecting malicious software. no starch press (2012)
 22. Sysdig: Security for containers, kubernetes, and cloud. <https://sysdig.com/>
 23. Taylor, M., Vaidya, R., Davidson, D., De Carli, L., Rastogi, V.: Defending against package typosquatting. In: Network and System Security: 14th International Conference, NSS 2020, Melbourne, VIC, Australia, November 25–27, 2020, Proceedings 14, pp. 112–131. Springer (2020)
 24. Tech, G.C.: Sandboxing your containers with gvisor (cloud next '18) (2018). URL <https://www.youtube.com/watch?v=kxUZ4IVFuVo>
 25. Tessian: What is a software supply chain attack? (2023). URL <https://www.tessian.com/blog/what-is-a-software-supply-chain-attack>

26. thehackernews: Malicious pypi packages slip whitesnake infostealer malware onto windows machines. <https://thehackernews.com/2024/01/malicious-pypi-packages-slip-whitesnake.html> (2024)
27. Vu, D.L.: A fork of bandit tool with patterns to identifying malicious python code (2020). URL <https://github.com/lyvd/bandit4mal>
28. Vu, D.L., Massacci, F., Pashchenko, I., Plate, H., Sabetta, A.: Lastpymile: identifying the discrepancy between sources and packages. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 780–792 (2021)
29. Vu, D.L., Newman, Z., Meyers, J.S.: Bad snakes: Understanding and improving python package index malware scanning. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 499–511. IEEE (2023)
30. Vu, D.L., Pashchenko, I., Massacci, F., Plate, H., Sabetta, A.: Typosquatting and combosquatting attacks on the python ecosystem. In: 2020 IEEE European Symposium on Security and Privacy Workshops (euros&pw), pp. 509–514. IEEE (2020)
31. Warehouse: Malware checks. <https://warehouse.readthedocs.io/development/malware-checks/#malware-checks> (2020)
32. Zahan, N., Zimmermann, T., Godefroid, P., Murphy, B., Maddila, C., Williams, L.: What are weak links in the npm supply chain? In: 2022 IEEE/ACM 44th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 331–340. IEEE (2022)